

Q1. Second Largest Element

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        int[] arr = {12, 45, 7, 31, 28};
        int largest = Integer.MIN_VALUE;
        int second = Integer.MIN_VALUE;
        for (int x : arr) {
            if (x > largest) {
                second = largest;
                largest = x;
            } else if (x > second && x != largest) {
                second = x;
            }
        }
        System.out.println("Second Largest: " + second);
    }
}
```

Output

Input: {12, 45, 7, 31, 28}

Output:

Second Largest: 31

Q2. Find Duplicate Elements

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        int[] arr = {5, 8, 5, 2, 8, 9, 8};
        System.out.println("Duplicate elements:");
        for (int i = 0; i < arr.length; i++) {
            boolean alreadyPrinted = false;
            for (int k = 0; k < i; k++) {
                if (arr[k] == arr[i]) {
                    alreadyPrinted = true;
                    break;
                }
            }
            if (alreadyPrinted) continue;
            for (int j = i + 1; j < arr.length; j++) {
                if (arr[i] == arr[j]) {
                    System.out.println(arr[i]);
                    break;
                }
            }
        }
    }
}
```

Output

Input: {5, 8, 5, 2, 8, 9, 8}

Output:

Duplicate elements:

5

8

Q3. Remove Duplicate Elements

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        int[] arr = {4, 7, 4, 2, 7, 9, 2};
        System.out.println("Array after removing duplicates:");
        for (int i = 0; i < arr.length; i++) {
            boolean duplicate = false;
            for (int j = 0; j < i; j++) {
                if (arr[i] == arr[j]) {
                    duplicate = true;
                    break;
                }
            }
            if (!duplicate)
                System.out.print(arr[i] + " ");
        }
    }
}
```

Output

Input: {4, 7, 4, 2, 7, 9, 2}

Output:

Array after removing duplicates:

4 7 2 9

Q4. Frequency of Array Elements

```

import java.util.*;
public class Main {
    public static void main(String[] args) {
        int[] arr = {4, 6, 4, 2, 6, 4, 9};
        for (int i = 0; i < arr.length; i++) {
            boolean counted = false;
            for (int j = 0; j < i; j++) {
                if (arr[i] == arr[j]) {
                    counted = true;
                    break;
                }
            }
            if (counted) continue;
            int count = 0;
            for (int x : arr) {
                if (x == arr[i]) count++;
            }
            System.out.println(arr[i] + " occurs " + count + " times");
        }
    }
}

```

Output

Input: {4, 6, 4, 2, 6, 4, 9}
Output:
4 occurs 3 times
6 occurs 2 times
2 occurs 1 times
9 occurs 1 times

Q5. Find Missing Number

```

public class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 5, 6, 7};
        int n = 7;
        int expected = n * (n + 1) / 2;
        int actual = 0;
        for (int x : arr)
            actual += x;
        System.out.println("Missing Number: " + (expected - actual));
    }
}

```

Output

Input: {1, 2, 3, 5, 6, 7}, n = 7
Output:
Missing Number: 4

Q6. Move All Zeros to the End

```

public class Main {
    public static void main(String[] args) {
        int[] arr = {0, 4, 0, 7, 2, 0, 9};
        int index = 0;
        for (int x : arr) {
            if (x != 0)
                arr[index++] = x;
        }
        while (index < arr.length)
            arr[index++] = 0;
        for (int x : arr)
            System.out.print(x + " ");
    }
}

```

Output

Input: {0, 4, 0, 7, 2, 0, 9}
Output:
4 7 2 9 0 0 0

Q7. Rotate Array by K Positions

```

public class Main {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 40, 50, 60};
        int k = 3;
        k = k % arr.length;
        reverse(arr, 0, arr.length - 1);
        reverse(arr, 0, k - 1);
        reverse(arr, k, arr.length - 1);
        for (int x : arr)
            System.out.print(x + " ");
    }
    static void reverse(int[] arr, int left, int right) {
        while (left < right) {
            int temp = arr[left];

```

```

        arr[left] = arr[right];
        arr[right] = temp;
        left++;
        right--;
    }
}
}

```

Output

Input: {10, 20, 30, 40, 50, 60}, k = 3
Output:
40 50 60 10 20 30

Q8. Maximum Subarray Sum

```

public class Main {
    public static void main(String[] args) {
        int[] arr = {-3, 5, -2, 6, -1, 4, -8};
        int current = arr[0];
        int max = arr[0];
        for (int i = 1; i < arr.length; i++) {
            current = Math.max(arr[i], current + arr[i]);
            max = Math.max(max, current);
        }
        System.out.println("Maximum Subarray Sum: " + max);
    }
}

```

Output

Input: {-3, 5, -2, 6, -1, 4, -8}
Output:
Maximum Subarray Sum: 12

Q9. Find Pair with Given Sum

```

public class Main {
    public static void main(String[] args) {
        int[] arr = {1, 8, 6, 3, 5, 4};
        int target = 9;
        for (int i = 0; i < arr.length; i++) {
            for (int j = i + 1; j < arr.length; j++) {
                if (arr[i] + arr[j] == target)
                    System.out.println("(" + arr[i] + ", " + arr[j] + ")");
            }
        }
    }
}

```

Output

Input: {1, 8, 6, 3, 5, 4}, target = 9
Output:
(1, 8)
(6, 3)
(5, 4)

Q10. Merge Two Sorted Arrays

```

public class Main {
    public static void main(String[] args) {
        int[] a = {2, 5, 8};
        int[] b = {1, 3, 7, 9};
        int i = 0, j = 0;
        while (i < a.length && j < b.length) {
            if (a[i] < b[j])
                System.out.print(a[i++] + " ");
            else
                System.out.print(b[j++] + " ");
        }
        while (i < a.length) System.out.print(a[i++] + " ");
        while (j < b.length) System.out.print(b[j++] + " ");
    }
}

```

Output

Input: a = {2, 5, 8}, b = {1, 3, 7, 9}
Output:
1 2 3 5 7 8 9

Q11. Check Whether Two Strings Are Anagrams

```

import java.util.*;
public class Main {
    public static void main(String[] args) {
        String s1 = "triangle";
        String s2 = "integral";
        char[] a = s1.toCharArray();
    }
}

```

```

        char[] b = s2.toCharArray();
        Arrays.sort(a);
        Arrays.sort(b);
        if (Arrays.equals(a, b))
            System.out.println("Anagram");
        else
            System.out.println("Not Anagram");
    }
}

```

Output

Input: "triangle", "integral"
Output:
Anagram

Q12. First Non-Repeating Character

```

public class Main {
    public static void main(String[] args) {
        String str = "aabbcdce";
        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            int count = 0;
            for (int j = 0; j < str.length(); j++) {
                if (str.charAt(j) == ch)
                    count++;
            }
            if (count == 1) {
                System.out.println("First non-repeating character: " + ch);
                break;
            }
        }
    }
}

```

Output

Input: "aabbcdce"
Output:
First non-repeating character: c

Q13. Remove Duplicate Characters

```

public class Main {
    public static void main(String[] args) {
        String str = "mississippi";
        String result = "";
        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            if (result.indexOf(ch) == -1)
                result += ch;
        }
        System.out.println(result);
    }
}

```

Output

Input: "mississippi"
Output:
mispi

Q14. Find Longest Word

```

public class Main {

    public static void main(String[] args) {
        String sentence = "Java makes programming easier";
        String[] words = sentence.split(" ");
        String longest = "";
        for (String word : words) {
            if (word.length() > longest.length())
                longest = word;
        }
        System.out.println("Longest word: " + longest);
    }
}

```

Output

Input: "Java makes programming easier"
Output:
Longest word: programming

Q15. Reverse Words in a Sentence

```

public class Main {
    public static void main(String[] args) {
        String sentence = "Java is very useful";
    }
}

```

```

        String[] words = sentence.split(" ");
        for (int i = words.length - 1; i >= 0; i--)
            System.out.print(words[i] + " ");
    }
}

```

Output

Input: "Java is very useful"

Output:

useful very is Java

Q16. Count Vowels, Consonants, Digits and Special Characters

```

public class Main {
    public static void main(String[] args) {
        String str = "Hello123!@";
        int vowels = 0, consonants = 0, digits = 0, special = 0;
        for (char ch : str.toCharArray()) {
            if (Character.isDigit(ch))
                digits++;
            else if (Character.isLetter(ch)) {
                if ("aeiouAEIOU".indexOf(ch) != -1)
                    vowels++;
                else
                    consonants++;
            } else {
                special++;
            }
        }
        System.out.println("Vowels: " + vowels);
        System.out.println("Consonants: " + consonants);
        System.out.println("Digits: " + digits);
        System.out.println("Special Characters: " + special);
    }
}

```

Output

Input: "Hello123!@"

Output:

Vowels: 2

Consonants: 3

Digits: 3

Special Characters: 2

Q17. Palindrome Number Without String

```

public class Main {
    public static void main(String[] args) {
        int num = 1221;
        int original = num;
        int reverse = 0;
        while (num > 0) {
            int digit = num % 10;
            reverse = reverse * 10 + digit;
            num /= 10;
        }
        if (original == reverse)
            System.out.println("Palindrome");
        else
            System.out.println("Not Palindrome");
    }
}

```

Output

Input: 1221

Output:

Palindrome

Q18. Armstrong Numbers in a Range

```

public class Main {
    public static void main(String[] args) {
        int start = 1, end = 500;
        for (int n = start; n <= end; n++) {
            int temp = n;
            int sum = 0;
            while (temp > 0) {
                int digit = temp % 10;
                sum += digit * digit * digit;
                temp /= 10;
            }
            if (sum == n)
                System.out.print(n + " ");
        }
    }
}

```

Output

Input: start = 1, end = 500

Output:

1 153 370 371 407

Q19. Prime Numbers in a Range

```
public class Main {
    public static void main(String[] args) {
        int start = 20, end = 40;
        int count = 0;
        System.out.println("Prime numbers:");
        for (int n = start; n <= end; n++) {
            if (n < 2) continue;
            boolean prime = true;
            for (int i = 2; i * i <= n; i++) {
                if (n % i == 0) {
                    prime = false;
                    break;
                }
            }
            if (prime) {
                System.out.print(n + " ");
                count++;
            }
        }
        System.out.println("
Count: " + count);
    }
}
```

Output

Input: start = 20, end = 40

Output:

Prime numbers:

23 29 31 37

Count: 4

Q20. Decimal to Binary

```
public class Main {
    public static void main(String[] args) {
        int num = 18;
        int binary = 0;
        int place = 1;
        while (num > 0) {
            int remainder = num % 2;
            binary = binary + remainder * place;
            place *= 10;
            num /= 2;
        }
        System.out.println("Binary: " + binary);
    }
}
```

Output

Input: 18

Output:

Binary: 10010

Q21. Two Sum

```
#include <stdio.h>
int main() {
    int nums[] = {3, 7, 2, 8};
    int n = 4, target = 10;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (nums[i] + nums[j] == target) {
                printf("[%d, %d]", i, j);
                return 0;
            }
        }
    }
    return 0;
}
```

Output

Input: nums = [3, 7, 2, 8], target = 10

Output:

[0, 1]

Q22. Remove Duplicates from Sorted Array

```
#include <stdio.h>
int main() {
```

```

int nums[] = {1, 1, 2, 2, 3, 4, 4};
int n = 7;
int k = 1;
for (int i = 1; i < n; i++) {
    if (nums[i] != nums[k - 1])
        nums[k++] = nums[i];
}
printf("k = %d\n", k);
printf("Array: ");
for (int i = 0; i < k; i++)
    printf("%d ", nums[i]);
return 0;
}

```

Output

Input: [1, 1, 2, 2, 3, 4, 4]
Output:
k = 4
Array: 1 2 3 4

Q23. Move Zeroes

```

#include <stdio.h>
int main() {
    int nums[] = {0, 5, 0, 2, 8, 0};
    int n = 6, pos = 0;
    for (int i = 0; i < n; i++) {
        if (nums[i] != 0)
            nums[pos++] = nums[i];
    }
    while (pos < n)
        nums[pos++] = 0;
    for (int i = 0; i < n; i++)
        printf("%d ", nums[i]);
    return 0;
}

```

Output

Input: [0, 5, 0, 2, 8, 0]
Output:

5 2 8 0 0 0

Q24. Merge Sorted Array

```

#include <stdio.h>
int main() {
    int nums1[] = {1, 4, 7, 0, 0, 0};
    int nums2[] = {2, 5, 6};
    int m = 3, n = 3;
    int i = m - 1, j = n - 1, k = m + n - 1;
    while (i >= 0 && j >= 0) {
        if (nums1[i] > nums2[j])
            nums1[k--] = nums1[i--];
        else
            nums1[k--] = nums2[j--];
    }
    while (j >= 0)
        nums1[k--] = nums2[j--];
    for (i = 0; i < m + n; i++)
        printf("%d ", nums1[i]);
    return 0;
}

```

Output

Input: nums1 = [1,4,7,0,0,0], nums2 = [2,5,6]
Output:
1 2 4 5 6 7

Q25. Reverse String

```

#include <stdio.h>
int main() {
    char s[] = {'c', 'o', 'd', 'e', '\0'};
    int left = 0, right = 3;
    while (left < right) {
        char temp = s[left];
        s[left] = s[right];
        s[right] = temp;
        left++;
        right--;
    }
    printf("%s", s);
    return 0;
}

```

Output

Input: ["c","o","d","e"]
Output:
edoc

Q26. Single Number

```
#include <stdio.h>
int main() {
    int nums[] = {6, 3, 4, 3, 4};
    int n = 5;
    int result = 0;
    for (int i = 0; i < n; i++)
        result ^= nums[i];
    printf("Single number: %d", result);
    return 0;
}
```

Output

Input: [6,3,4,3,4]
Output:
Single number: 6

Q27. Reshape the Matrix

```
#include <stdio.h>
int main() {
    int mat[2][2] = {{1, 2}, {3, 4}};
    int r = 4, c = 1;
    if (2 * 2 != r * c) {
        printf("Reshape not possible");
        return 0;
    }
    int index = 0;
    int result[4][1];
    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 2; j++)
            result[index++][0] = mat[i][j];
    for (int i = 0; i < r; i++)
        printf("[%d]\n", result[i][0]);
    return 0;
}
```

Output

Input: mat = [[1,2],[3,4]], r = 4, c = 1
Output:
[1]
[2]
[3]
[4]

Q28. Middle of Linked List

```
#include <stdio.h>
struct Node {
    int data;
    struct Node *next;
};
int main() {
    struct Node n1 = {10, NULL};
    struct Node n2 = {20, NULL};
    struct Node n3 = {30, NULL};
    struct Node n4 = {40, NULL};
    struct Node n5 = {50, NULL};
    n1.next = &n2;
    n2.next = &n3;
    n3.next = &n4;
    n4.next = &n5;
    struct Node *slow = &n1;
    struct Node *fast = &n1;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    printf("Middle node: %d", slow->data);
    return 0;
}
```

Output

Input: 10 -> 20 -> 30 -> 40 -> 50
Output:
Middle node: 30

Q29. Matrix Diagonal Sum

```
#include <stdio.h>
int main() {
```

```

int mat[4][4] = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12},
    {13, 14, 15, 16}
};
int n = 4, sum = 0;
for (int i = 0; i < n; i++) {
    sum += mat[i][i];
    sum += mat[i][n - 1 - i];
}
printf("Diagonal sum: %d", sum);
return 0;
}

```

Output

Input: 4 x 4 matrix shown in code

Output:

Diagonal sum: 68

Q30. Richest Customer Wealth

```

#include <stdio.h>
int main() {
    int accounts[3][3] = {
        {2, 8, 1},
        {5, 1, 4},
        {3, 7, 2}
    };
    int max = 0;
    for (int i = 0; i < 3; i++) {
        int sum = 0;
        for (int j = 0; j < 3; j++)

            sum += accounts[i][j];
        if (sum > max)
            max = sum;
    }
    printf("Richest customer wealth: %d", max);
    return 0;
}

```

Output

Input: [[2,8,1],[5,1,4],[3,7,2]]

Output:

Richest customer wealth: 12